

PADP (18CS73)

Computer Architecture: A Quantitative Approach, Fifth Edition

UNIT 2 – Multiprocessors and Thread-Level Parallelism (Chapter 5)

Structured Exam-Revision Notes

Based on slides by Dr. Minal Moharir (Hennessy & Patterson, Ch. 5)

Contents

- Part 1 – Introduction: Thread-Level Parallelism & Types of Multiprocessors
- Part 2 – Cache Coherence & Consistency (Concepts)
- Part 3 – Snoopy Coherence Protocols (MSI, MESI, Extensions)
- Part 4 – Performance of Symmetric Shared-Memory Multiprocessors
- Part 5 – Distributed Shared Memory & Directory-Based Coherence
- Part 6 – Synchronization (Locks, Atomic Operations, LL/SC)
- Part 7 – Models of Memory Consistency (SC, TSO, XC, Relaxed)
- Quick Revision – Comparison Tables & Key Terms

PART 1 – Introduction: Thread-Level Parallelism

1.1 Thread-Level Parallelism (TLP)

- **Thread-Level parallelism** has **multiple program counters** and **uses the MIMD model**.
- Targeted for **tightly-coupled shared-memory multiprocessors**.
- For **n processors**, **need n threads**.

DEFINITION

Grain size – the amount of computation assigned to each thread.

- Threads can be used for **data-level parallelism**, but the **overheads may outweigh the benefit**.

1.2 Types of Shared-Memory Multiprocessors

	Symmetric Multiprocessors (SMP)	Distributed Shared Memory (DSM)
Cores	Small number of cores	Larger scale
Memory	Share single memory with uniform memory latency	Memory distributed among processors
Latency	Uniform (also called UMA)	Non-uniform memory access/latency (NUMA)
Connection	Shared bus / shared cache; private caches per processor (one or more levels)	Processors connected via direct (switched) and non-direct (multi-hop) interconnection networks

- **SMP structure:** processors, each with one or more levels of **private cache**, a **shared cache**, **main memory** and I/O system.
- **DSM structure:** each node = multicore MP + memory + I/O; nodes joined by an **interconnection network**.

1.3 Role of Caches

- Caches serve to: **increase bandwidth versus bus/memory; reduce latency of access**; valuable for **both private data and shared data**.
- Question raised: **what about cache coherence and consistency?**

SECTION SUMMARY

TLP = MIMD with multiple PCs, needs n threads for n processors. SMP = small, single memory, uniform latency; DSM = distributed memory, NUMA, interconnection network. Caches help but create the coherence/consistency problem.

PART 2 – Cache Coherence & Consistency (Concepts)

2.1 The Cache Coherence Problem

- Processors may see different values through their caches:

Time	Event	Cache contents for processor A	Cache contents for processor B	Memory contents for location X
0				1
1	Processor A reads X	1		1
2	Processor B reads X	1	1	1
3	Processor A stores 0 into X	0	1 (stale!)	0

KEY POINT

After time 3, processor B still holds the **old value 1** in its cache, while memory and A hold 0 – B sees a stale value.

2.2 Coherence vs Consistency

Coherence	Consistency
All reads by any processor must return the most recently written value. Writes to the same location by any two processors are seen in the same order by all processors.	Determines when a written value will be returned by a read. If a processor writes location A followed by location B, any processor that sees the new value of B must also see the new value of A.

2.3 What does Coherence mean?

- Informally:** “Any read must return the most recent write” – **too strict and too difficult to implement.**
- Better:** “Any write must eventually be seen by a read”; **all writes are seen in proper order (“serialization”).**

Two rules to ensure this

- “If P writes x and P1 reads it, P’s write will be seen by P1 **if the read and write are sufficiently far apart.**”
- Writes to a single location are serialized:** seen in one order – the latest write will be seen. Otherwise one could see writes in illogical order (could see an **older value after a newer value**).

Further points on coherence vs consistency

- The goal of cache coherence is to **make caches in multi-core systems invisible.**
- Coherence deals with **one block at a time** and is silent on the interaction of accesses to multiple cache blocks.
- It is possible to implement a memory system **without coherence and even without caches.**
- Cache coherence is not equal to memory consistency** – use coherence as a black box that implements **Single-Writer-Multiple-Readers** invariants.

2.4 Enforcing Coherence

- Coherent caches provide:
 - **Migration** – movement of data
 - **Replication** – multiple copies of data

Cache coherence protocols

Protocol	Idea	Notes
Directory based	Sharing status of each block kept in one location (1 centralized place, logically)	Distributed memory $\Rightarrow$ distributed directory for scalability (avoids bottlenecks). Sends point-to-point requests to processors via network. Scales better than snooping . Actually existed before snooping-based schemes.
Snooping	Each core tracks sharing status of each block	Send all requests for data to all processors ; processors snoop to see if they have a copy and respond accordingly. Requires broadcast , since caching information is at processors. Works well with bus (natural broadcast medium) . Dominates for small-scale machines (most of the market) .

SECTION SUMMARY

Coherence = reads return latest write + writes to a location serialized. Consistency = when/in what order writes become visible across locations. Two hardware solutions: snooping (broadcast, small machines) and directory (point-to-point, scalable).

PART 3 – Snoopy Coherence Protocols

3.1 Write Invalidate vs Write Update

	Write Invalidate	Write Update
On a write	Invalidate all other copies	Update all copies
Serialization	Use bus itself to serialize – the write cannot complete until bus access is obtained	—
Result (P1 writes X1)	P1 holds X1; caches of P2, P3 become I (invalid); memory holds X1	All caches (P1, P2, P3) hold X1

Write-invalidate example

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1	(invalidated)	0
Processor B reads X	Cache miss for X	1	1	1

Locating an item when a read miss occurs

- In a **write-back cache**, the **updated value must be sent to the requesting processor**.
- Cache lines are marked as **shared** or **exclusive/modified**.
- **Only writes to shared lines need an invalidate broadcast**; after this, the line is marked as **exclusive**.
- Hardware: each private cache has an **n-bit coherence state** per block; a **bus snoop** watches cache transactions. A **controller updates the state of each cache block** in response to **processor and snoop events** and updates the bus transaction (state-transition mechanism for each cache block).

3.2 The 3-State (MSI) Write-Invalidate Protocol

State	Meaning
Invalid (I)	Block not present / not valid in this cache
Shared (S, read only)	Block may be in several caches; read only
Exclusive / Modified (M, read/write)	Only this cache has the block; read/write (memory out of date)

State transitions based on requests from the CPU

Current state	CPU event	Action	Next state
Invalid	CPU read	Place read miss on bus	Shared
Invalid	CPU write	Place write miss on bus	Exclusive
Shared	CPU read hit	Read data in local cache	Shared

Current state	CPU event	Action	Next state
Shared	CPU read miss	Place read miss on bus	Shared
Shared	CPU write (hit)	Place invalidate on bus	Exclusive
Shared	CPU write miss	Place write miss on bus	Exclusive
Exclusive	CPU read hit / CPU write hit	Use local cache	Exclusive
Exclusive	CPU read miss	Write-back block; place read miss on bus	Shared
Exclusive	CPU write miss	Write-back cache block; place write miss on bus	Exclusive

- [The slide series builds this diagram step by step: first the read transitions (slide 19), then adds writes (slides 20–21). The table above merges them.]

Cache behaviour summary table (requests from processor and bus)

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or ownership misses , since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared.
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

3.3 Complications and Extensions

Complications for the basic MSI protocol

- **Operations are not atomic** – e.g. detect miss, acquire bus, receive a response.
 - Creates possibility of **deadlock and races**.
 - **One solution:** processor that sends invalidate can **hold bus until other processors receive the invalidate**.

Extensions

- **MESI protocol:** add **Exclusive state** to indicate a **clean block in only one cache** – prevents needing to write invalidate on a write.
- **Owned state (MOESI).**

Scaling snooping – coherence protocol extensions

- **Shared memory bus and snooping bandwidth is the bottleneck** for scaling symmetric multiprocessors. Remedies:
 - **Duplicating tags**
 - **Place directory in outermost cache**
 - **Use crossbars or point-to-point networks with banked memory** (processors with private caches → interconnection network → shared cache banks 0–3 → memory and I/O)

AMD Opteron (example)

- **Memory directly connected to each multicore chip** in a **NUMA-like organization**.
- Implements the coherence protocol using **point-to-point links**.
- Uses **explicit acknowledgements** to order operations.

MNEMONIC

MSI = Modified, Shared, Invalid. **MESI** adds Exclusive (clean, single copy). **MOESI** adds Owned.

SECTION SUMMARY

Snooping: all caches watch the bus. Write invalidate is the common approach (write update = send new value to all). MSI has 3 states per block; writes to Shared blocks need an invalidate. Bus bandwidth limits scaling – use duplicated tags, outer-cache directory, crossbars/point-to-point.

PART 4 – Performance of Symmetric Shared-Memory Multiprocessors

- Coherence influences cache miss rate.

4.1 Coherence Misses

Type	When it occurs
True sharing misses	Write to shared block (transmission of invalidation) Read an invalidated block
False sharing misses	Read an unmodified word in an invalidated block (the block was invalidated because a different word in it was written)

KEY POINT

True sharing = the same **word** is really shared. False sharing = different words that merely sit in the **same block**.

4.2 Performance Study: Commercial Workload

The slides show four charts. [Observations read from the charts:]

Chart	Axis / variable	Observation
Normalized execution time (slide 27)	L3 cache size: 1, 2, 4, 8 MB	Time falls as L3 grows (mainly the memory-access component shrinks); instruction execution and L2/L3 access stay roughly constant. Components: idle, PAL code, memory access, L2/L3 cache access, instruction execution.
Memory cycles per instruction (slide 28)	Cache size: 1, 2, 4, 8 MB	Total falls with bigger cache (instruction, capacity/conflict, compulsory misses drop) but true-sharing component stays about the same .
Memory cycles per instruction (slide 29)	Processor count: 1, 2, 4, 6, 8	Memory cycles per instruction rise with processor count , mainly because true-sharing misses grow .
Misses per 1000 instructions (slide 30)	Block size: 32, 64, 128, 256 bytes	Misses per 1000 instructions fall as block size increases ; true-sharing misses decrease with larger blocks.

- Miss categories in the charts: **Instruction, Capacity/conflict, Compulsory, False sharing, True sharing**.

SECTION SUMMARY

Coherence adds true- and false-sharing misses. A bigger cache does not remove true-sharing misses; more processors increase them.

PART 5 – Distributed Shared Memory & Directory-Based Coherence

5.1 Directory Protocols – Basics

- **Directory keeps track of every block:** which caches have each block; **dirty status** of each block.
- **Implement in shared L3 cache:** keep a **bit vector of size = number of cores** for each block in L3 – **not scalable beyond shared L3**.
- **Implement in a distributed fashion:** each node (multicore processor + caches + memory + I/O) has its own **directory**, all connected by an **interconnection network**.
- **Maintain the state information explicitly:** associated with each memory block; records the state of each block in cache. Directory entry = **presence bits p[1..n]** + **dirty bit**.
- **On a cache miss, the node communicates with the directory:** determines where the valid cache copies are (if any); determines what further actions need to be taken; performs a set of transactions to keep the directory up-to-date.

Term	Meaning
Home node	Node in whose memory chunk the block is allocated
Owner node	Node that currently holds a valid copy of the block
CC	Cache controller (one per node)

5.2 Directory Block States

State	Meaning
Shared	One or more nodes have the block cached, value in memory is up-to-date . Directory stores the set of node IDs .
Uncached	No node has a copy of the block
Modified	Exactly one node has a copy of the cache block, value in memory is out-of-date . Directory stores the owner node ID .

- The **directory maintains block states and sends invalidation messages**.

5.3 Cache-Side 3-State Protocol (write-back, invalidation-based)

State	Event	Action	Next state
Invalid	CPU read miss	Send read miss message	Shared
Invalid	CPU write miss	Send write miss message	Modify
Shared	CPU read hit	—	Shared
Shared	CPU read miss	Send read miss message	Shared
Shared	CPU write hit	Send invalidate message	Modify
Shared	CPU write miss	Send write miss message	Modify
Shared	Invalidate (from directory)	—	Invalid
Modify	CPU read hit / write hit	—	Modify
Modify	CPU read miss	WB block; send read miss	Shared

State	Event	Action	Next state
Modify	CPU write miss	WB block; send write miss	Modify
Modify	Fetch (from directory)	WB block	Shared
Modify	Fetch/invalidate (from directory)	WB block	Invalid

5.4 Messages

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

5.5 Directory Actions by Block State

Block state	Event	Directory action	New state
Uncached	Read miss	Requesting node is sent the requested data and is made the only sharing node	Shared
Uncached	Write miss	Requesting node is sent the requested data and becomes the sharing node	Exclusive
Shared	Read miss	Requesting node is sent the data from memory; node is added to sharing set	Shared
Shared	Write miss	Requesting node is sent the value; all nodes in the sharing set are sent invalidate messages ; sharing set only contains the requesting node	Exclusive
Exclusive	Read miss	Owner is sent a data fetch message ; owner sends data to the directory; data written back to memory ; sharers set contains old owner and requestor	Shared
Exclusive	Data write-back	Block becomes uncached, sharer set is empty	Uncached

Block state	Event	Directory action	New state
Exclusive	Write miss	Message sent to old owner to invalidate and send the value to the directory ; requestor becomes new owner	Exclusive

Directory state diagram (slide 35) in short

- **Uncached → Shared**: read miss; data value reply; Sharers = {P}.
- **Uncached → Modify**: write miss; data value reply; Sharers = {P}.
- **Shared → Shared**: read miss; data value reply; Sharers = Sharers + {P}.
- **Shared → Modify**: write miss; invalidate; Sharers = {P}; data value reply.
- **Modify → Shared**: read miss; fetch; data value reply; Sharers = Sharers + {P}.
- **Modify → Modify**: write miss; fetch/invalidate; data value reply; Sharers = {P}.
- **Modify → Uncached**: data write-back; Sharers = {}.

Snooping	Directory
Broadcast on a bus; every cache watches	Point-to-point messages to home directory / specific nodes
Sharing state held in each cache	Sharing state held in the directory (presence bits + dirty bit)
Good for small machines; bus bandwidth bottleneck	Scales better; suits distributed memory (NUMA)

SECTION SUMMARY

Directory = per-block record of state (Uncached/Shared/Modified) and sharers. Home node owns the memory; owner node holds the modified copy. Misses go to the home directory, which sends fetch / invalidate messages point-to-point.

PART 6 – Synchronization

6.1 Why and Types

- To ensure consistency of shared data structures.

Type of synchronization	Mechanism
Mutual exclusion	lock–unlock pairs
Point-to-point event synchronization	flags
Global event synchronization	barriers

- **Hardware** – speed. **Software** – cost, flexibility, and adaptability.
- A lock ensures that only one process enters the critical section. **Hardware locks:** lock lines on the bus; lock locations in memory; lock registers.

6.2 Simple Software Lock (and its flaw)

```
lock:  ld  register, location    /* copy location to register */
      cmp register, #0          /* compare with 0 */
      bnz lock                  /* if not 0, try again */
      st  location, #1          /* store 1 to mark it locked */
      ret                       /* return control to caller */

unlock: st  location, #0         /* write 0 to location */
      ret                       /* return control to caller */
```

PROBLEM

Two processes can enter the critical section – the load/compare and the store are separate, so another process can sneak in between them. Hence **atomic operations** are needed.

6.3 Atomic Read-Modify-Write Primitives

- **Hardware primitives to implement synchronization:** atomic **test-and-set**, **swap**, **fetch-and-increment**, etc.
- Basic building blocks require **memory read and write in an uninterruptable instruction**.

Primitive	Operation
Atomic exchange / Swap	Atomically swaps the values in a memory location and a register
Test-and-set	Value in a memory location is read into a specified register ; constant 1 is stored into the location atomically ; successful if the value loaded into the register is 0 . (Other constants could be used instead of 0 and 1.) “Sets under condition.”
Fetch & op (fetch&increment, fetch&decrement, fetch&add)	Atomically reads the current value of a location into a register and writes the new value into the location . Fetch-and-increment: reads original value from memory and increments it in memory.
Compare & swap	Three operands: a memory location, register to compare with, register to swap with. Not commonly supported by RISC instruction sets.

```
lock:  t&s  register, location
      bnz  register, lock    /* if not 0, try again */
```

```

        ret                /* return control to caller */

unlock: st    location, #0    /* write 0 to location */
        ret                /* return control to caller */

```

Examples in real architectures

Architecture	Support
IBM 370	Included atomic compare&swap for multiprogramming
x86	Any instruction can be prefixed with a lock modifier
SPARC	Atomic register-memory ops (swap, compare&swap)
MIPS, IBM Power	No atomic operations, but a pair of instructions (load-locked, store-conditional)

6.4 Load-Linked / Store-Conditional (LL/SC)

- **Implementing a single atomic memory operation is complex** – so use a **pair of instructions** to implement synchronization.
- **LL** reads a memory location (synchronization variable) into a register.
- Followed by arbitrary instructions to modify the register value.
- **SC** tries to write the data of a register to the memory location – **if and only if no other write to the location since the processor's LL**; indicated by condition codes or a return value.
- **If SC succeeds, all three steps happened atomically. If it fails, it does not write or generate invalidations.**
- [In other words: if the contents of the location specified by the LL are changed before the SC to the same address, the **store conditional fails**.]

```

lock:  ll    r1, location
        bnz   r1, lock        ; location is non-zero => already locked, retry
        sc    location, r2     ; try to store (r2 holds the locked value, 1)
        beqz  lock            ; SC failed => retry
        ret

unlock: st    location, #0
        ret

```

How LL/SC is implemented in hardware

- Requires a **lock flag** and a **lock address register** at each processor.
- An **LL operation** sets the **lock flag** and puts the **address of the read block in the lock address register**.
- **Incoming invalidation (or update) requests are matched against the lock address register**; a successful match **resets the lock flag**.
- **SC checks the lock flag** to know whether an intervening write has occurred.
- The lock flag can also be reset when: **the lock variable is replaced from the cache**; a **context switch** happens between LL and SC.
- [Slides 46–48 illustrate this: P1 executes LL r1, XYZ (flag = 1, address = XYZ); P2 writes XYZ, causing Invalidate XYZ, which resets P1's flag to 0; P1's later SC XYZ, r2 is **unsuccessful**.]

6.5 Implementing Locks – Spin Locks

DEFINITION

Spin lock – a lock where the processor **repeatedly tests (spins on) the lock variable** until it becomes available.

If no coherence (no cache)

```

        DADDUI R2,R0,#1
lockit: EXCH   R2,0(R1)      ;atomic exchange
        BNEZ  R2,lockit     ;already locked?
    
```

If coherence (spin on the cached copy)

```

lockit: LD     R2,0(R1)      ;load of lock
        BNEZ  R2,lockit     ;not available-spin
        DADDUI R2,R0,#1     ;load locked value
        EXCH  R2,0(R1)      ;swap
        BNEZ  R2,lockit     ;branch if lock wasn't 0
    
```

KEY POINT

Advantage of this scheme: reduces memory traffic. Processors spin on their **cached copy** (read hits, no bus traffic) and only attempt the atomic swap when the lock appears free.

Spin-lock example with coherence (three processors P0, P1, P2)

Step	P0	P1	P2	Coherence state of lock at end of step	Bus/directory activity
1	Has lock	Begins spin, testing if lock = 0	Begins spin, testing if lock = 0	Shared	Cache misses for P1 and P2 satisfied in either order. Lock state becomes shared.
2	Set lock to 0	(Invalidate received)	(Invalidate received)	Exclusive (P0)	Write invalidate of lock variable from P0.
3		Cache miss	Cache miss	Shared	Bus/directory services P2 cache miss; write-back from P0; state shared.
4		(Waits while bus/directory busy)	Lock = 0 test succeeds	Shared	Cache miss for P2 satisfied.
5		Lock = 0	Executes swap, gets cache miss	Shared	Cache miss for P1 satisfied.
6		Executes swap, gets cache miss	Completes swap: returns 0 and sets lock = 1	Exclusive (P2)	Bus/directory services P2 cache miss; generates invalidate; lock is exclusive.
7		Swap completes and returns 1, and sets lock = 1	Enter critical section	Exclusive (P1)	Bus/directory services P1 cache miss; sends invalidate and generates write-back from P2.
8		Spins, testing if lock = 0			None

SECTION SUMMARY

Synchronization = lock/unlock, flags, barriers. A plain load-compare-store lock is broken; use atomic exchange, test-and-set, fetch-and-increment, or LL/SC (MIPS, Power). Spinning on a cached copy before the swap reduces bus traffic.

PART 7 – Models of Memory Consistency

7.1 The Problem

Processor 1:	Processor 2:
A=0	B=0
...	...
A=1	B=1
if (B==0) ...	if (A==0) ...

- It should be **impossible for both if-statements to be evaluated as true.**
- Possible cause: **delayed write invalidate?**
- **Consistency:** specification of the **allowed behaviour of multithreaded programs executing with shared memory.**
- Implications for both programmer and system designer:
 - **Programmer** uses it to reason about possible results and correctness.
 - **System designer** can use it to constrain how much accesses can be reordered by compiler or hardware.

7.2 Sequential Consistency (SC)

DEFINITION

“A multiprocessor is **sequentially consistent** if the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program.” – **Lamport**

- Result of execution should be the same as long as: **accesses on each processor were kept in order; accesses on different processors were arbitrarily interleaved.**
- **To implement:** delay completion of all memory accesses until all invalidations caused by the access are completed – **reduces performance!**

Example: $x = y = 0$ initially

Core C1	Core C2
S1: x = NEW	S2: y = NEW
L1: r1 = y	L2: r2 = x

Outcome (r1, r2)	Allowed under SC?	Reason
(0, NEW)	✓ Yes	C1 runs S1, L1 before C2 runs S2, L2
(NEW, 0)	✓ Yes	C2 runs S2, L2 before C1 runs S1, L1
(NEW, NEW)	✓ Yes	Both stores happen before both loads
(0, 0)	✗ No (forbidden by SC)	Would need L1 before S2 and L2 before S1, while S1 before L1 and S2 before L2 – a cycle

Alternatives to enforcing SC in hardware

- **Program-enforced synchronization** to force the write on one processor to occur before the read on the other processor.
 - Requires a synchronization object for A and another for B: “**Unlock**” after write; “**Lock**” after read.

7.3 Write Buffers and (0, 0)

- Processors use **write buffers** to hold committed stores until the stores are written back.
- Write buffers are **not a problem for a single-core processor**.
- **Multi-core processors can produce the result $(r1, r2) = (0, 0)$, which is forbidden by SC.**

7.4 Total Store Order (TSO)

- **TSO requires that each core preserves the program order of its loads and stores:**

Order	SC	TSO
Load → Load	Kept	Kept
Load → Store	Kept	Kept
Store → Store	Kept	Kept
Store → Load	Kept	Omitted (relaxed)

- **All cores insert their loads and stores into the global order ($<_m$) respecting their program order ($<_p$):**

If $L(a) <_p L(b) \Rightarrow L(a) <_m L(b)$

If $L(a) <_p S(b) \Rightarrow L(a) <_m S(b)$

If $S(a) <_p S(b) \Rightarrow S(a) <_m S(b)$ */* Enable FIFO write buffer */*

- **Every load gets its value from the last store before it to the same address:**

Value of $L(a)$ = Value of $\text{Max}_{<_m} \{ S(a) \mid S(a) <_m L(a) \text{ or } S(a) <_p L(a) \}$ */* Need bypassing */*

EXAMPLE

TSO example: C1: S1: x = NEW; L1: r1 = x; L2: r2 = y. C2: S2: y = NEW; L3: r3 = y; L4: r4 = x. (Initially x = y = 0; assume r2 = r4 = 0.)

Each core's load of its **own** store is satisfied by **bypass** from the write buffer: $(r1, r3) = (\text{NEW}, \text{NEW})$.

The loads of the **other** core's variable still see 0: $(r2, r4) = (0, 0)$. Allowed in TSO, not in SC.

Synchronization example (flag and data)

Core C1	Core C2	Comments
S1: data1 = NEW S2: data2 = NEW S3: flag = SET	L1: r1 = flag B1: if (r1 != SET) goto L1 L2: r2 = data1 L3: r3 = data2	Initially data = 0, flag != SET. L1 & B1 may repeat many times.

- **Programmers' expected orders:** $S1 \rightarrow S3 \rightarrow L1 \text{ loads SET} \rightarrow L2$; $S2 \rightarrow S3 \rightarrow L1 \text{ loads SET} \rightarrow L3$.
- In addition, **SC and TSO also require $S1 \rightarrow S2$ and $L2 \rightarrow L3$ – not needed by the program for correct operation.**

7.5 Relaxed Consistency Model (XC)

- **XC assumes that a global order exists.**
- XC provides a **FENCE instruction** – FENCE neither specifies an address nor affects the order of memory operations at other cores.
- **XC's memory order is guaranteed to respect program order for:** Load → FENCE; Store → FENCE; FENCE → FENCE; FENCE → Load; FENCE → Store.
- **XC maintains TSO rules for ordering two accesses to the same address only** – loads can immediately see updates due to their own stores.

Core C1	Core C2	Comments
S1: data1 = NEW S2: data2 = NEW F1: FENCE S3: flag = SET	L1: r1 = flag B1: if (r1 != SET) goto L1 F2: FENCE L2: r2 = data1 L3: r3 = data2	Initially data = 0, flag != SET. L1 & B1 may repeat many times.

- **FENCES ensure that:** S1, S2 → F1 → S3 → L1 loads SET → F2 → L2, L3.

7.6 Relaxed Consistency Models – Rules

- **X → Y** means: operation X must complete before operation Y is done.
- **Sequential consistency requires:** R → W, R → R, W → R, W → W.

Relaxation	Name of model
Relax W → R	"Total store ordering"
Relax W → W	"Partial store order"
Relax R → W and R → R	"Weak ordering" and "release consistency"

- **Consistency model is multiprocessor specific.**
- Programmers will often implement **explicit synchronization**.
- **Speculation gives much of the performance advantage of relaxed models with sequential consistency.** Basic idea: if an invalidation arrives for a result that has not been committed, **use speculation recovery**.

MNEMONIC

Order of strictness: **SC** (strongest) → **TSO** (relax W→R) → **PSO** (also W→W) → **Weak ordering / Release consistency** (also R→R, R→W). More relaxed = faster hardware, more work for the programmer (fences/locks).

SECTION SUMMARY

SC forbids (0,0); write buffers can produce it. TSO relaxes Store→Load; XC relaxes everything except around FENCES and same-address accesses. Programmers use explicit synchronization; speculation recovers much of the lost performance.

QUICK REVISION – Comparison Tables & Key Terms

Key Terms

Term	One-line meaning
TLP	Thread-level parallelism: multiple PCs, MIMD, tightly-coupled shared memory
Grain size	Amount of computation assigned to each thread
SMP / UMA	Few cores, single memory, uniform latency
DSM / NUMA	Memory distributed among processors, non-uniform latency
Coherence	Reads return most recent write; writes to a location serialized
Consistency	When a written value is returned by a read; ordering across locations
Migration / Replication	Movement of data / multiple copies of data
Write invalidate	On write, invalidate all other copies
Write update	On write, update all copies
MSI / MESI / MOESI	Invalid-Shared-Modified; + Exclusive (clean, single copy); + Owned
True / False sharing miss	Same word shared / different words in same block
Home / Owner node	Node holding the memory / node holding the valid modified copy
LL / SC	Load-linked / store-conditional pair for atomic synchronization
Spin lock	Lock acquired by repeatedly testing the lock variable
FENCE	Instruction ordering memory operations around it (XC)

Consistency Models at a Glance

Model	Relaxed order(s)	Needs
Sequential consistency (SC)	None	All of $R \rightarrow W$, $R \rightarrow R$, $W \rightarrow R$, $W \rightarrow W$ kept
Total store order (TSO)	$W \rightarrow R$	Write buffer with bypassing
Partial store order	$W \rightarrow R$, $W \rightarrow W$	Explicit sync for write ordering
Weak ordering / release consistency	Also $R \rightarrow R$, $R \rightarrow W$	Synchronization operations / FENCE
XC	Everything except around FENCES and same-address accesses	FENCE instructions

Likely Exam Questions (from slide coverage)

1. Explain the cache coherence problem and what coherence means. Differentiate coherence and consistency.
2. Compare snooping and directory-based protocols. Explain write invalidate vs write update with an example.
3. Explain the MSI snooping protocol with its state-transition diagram. What are MESI and the Owned state?

4. Explain true sharing and false sharing misses.
5. Explain directory-based coherence: states, messages and actions for read/write miss.
6. Explain atomic exchange, test-and-set, fetch-and-increment and LL/SC; implement a spin lock.
7. Explain sequential consistency and total store order with the $(r1, r2) = (0, 0)$ example. What are relaxed consistency models?